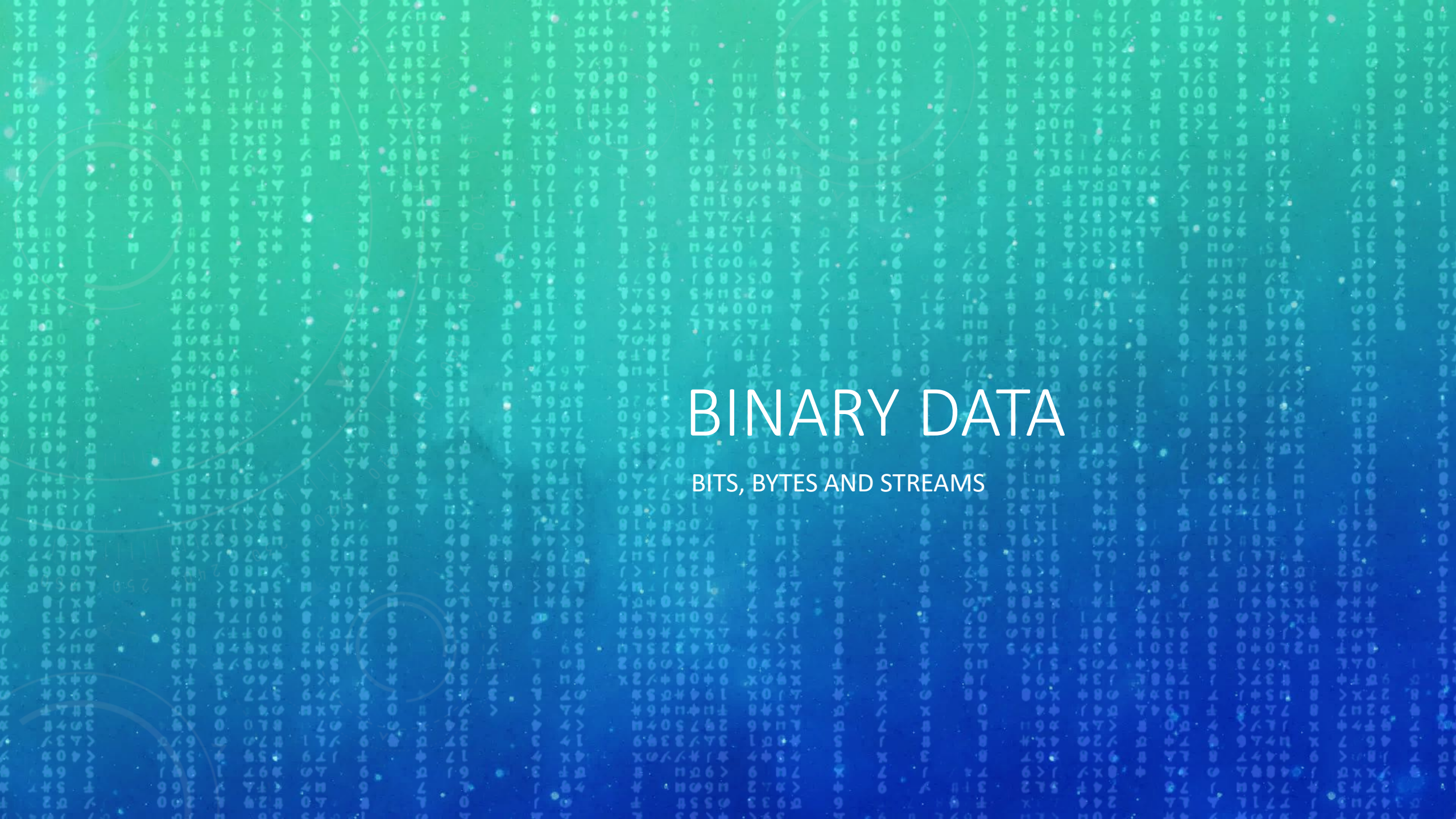


BINARY DATA

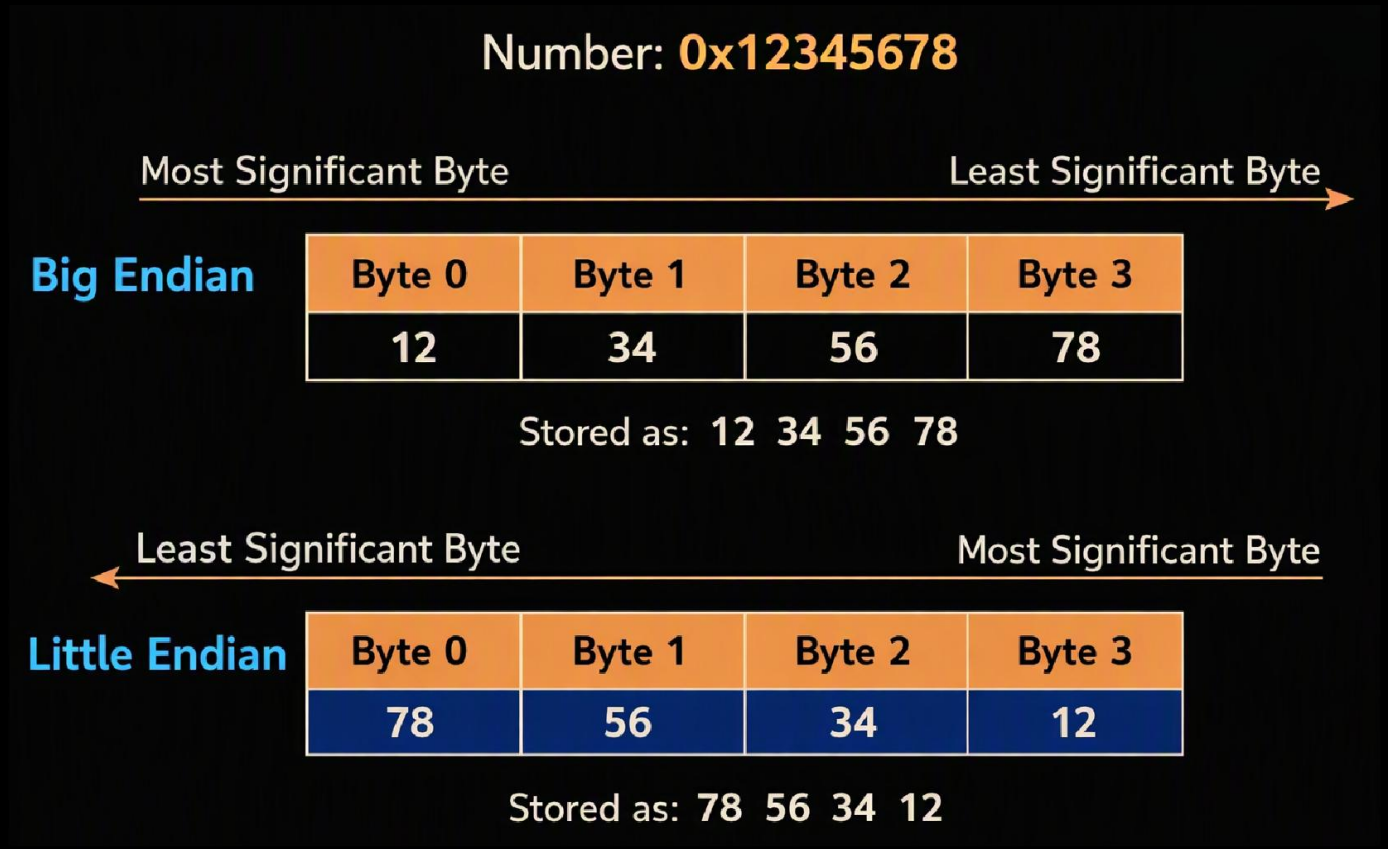
BITS, BYTES AND STREAMS

Text vs. Binary Data

- **Text data** refers to data that consists of readable characters, typically represented using encoding standards like ASCII or Unicode. It includes letters, numbers, punctuation, and whitespace. Text data is human-readable and is often stored in plain text files or databases.

- **Binary data** refers to data that is represented in binary (bits) and is not necessarily human-readable. It includes any type of data that is stored in a format that uses binary encoding, such as images, audio, video, and executable files.

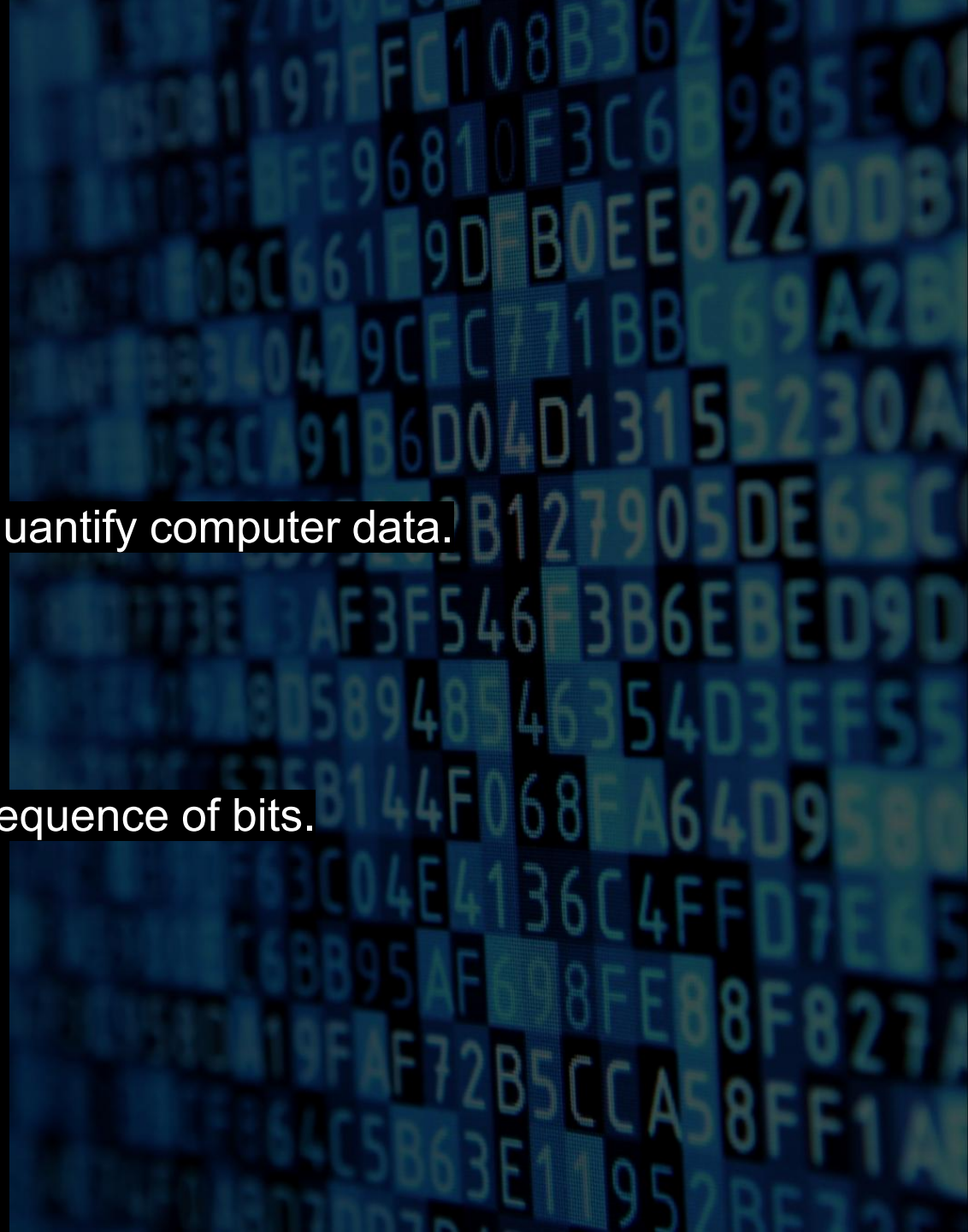
Endian



Big-endian and little-endian are terms that describe the order in which a sequence of bytes are stored in computer memory.

Bits, Bytes, Streams

- A bit is the smallest unit of measurement used to quantify computer data.
- A byte is a collection of 8-bits.
- A bitstream, also known as binary sequence, is a sequence of bits.
- A bytestream is a sequence of bytes.



characters			characters			characters			characters		
00	NULL	(Null character)	32	space	64	@	96	`	128	Ç	160	á	192	└	224	Ó
01	SOH	(Start of Header)	33	!	65	A	97	a	129	ü	161	í	193	┴	225	ß
02	STX	(Start of Text)	34	"	66	B	98	b	130	é	162	ó	194	┬	226	Ô
03	ETX	(End of Text)	35	#	67	C	99	c	131	â	163	ú	195	├	227	Ò
04	EOT	(End of Trans.)	36	$	68	D	100	d	132	ä	164	ñ	196	─	228	õ
05	ENQ	(Enquiry)	37	%	69	E	101	e	133	à	165	Ñ	197	┼	229	Õ
06	ACK	(Acknowledgement)	38	&	70	F	102	f	134	å	166	ª	198	ã	230	µ
07	BEL	(Bell)	39	'	71	G	103	g	135	ç	167	º	199	Ã	231	þ
08	BS	(Backspace)	40	(	72	H	104	h	136	ê	168	¿	200	╚	232	Þ
09	HT	(Horizontal Tab)	41	)	73	I	105	i	137	ë	169	®	201	╔	233	Ú
10	LF	(Line feed)	42	*	74	J	106	j	138	è	170	¬	202	╩	234	Û
11	VT	(Vertical Tab)	43	+	75	K	107	k	139	ï	171	½	203	╦	235	Ù
12	FF	(Form feed)	44	,	76	L	108	l	140	î	172	¼	204	╠	236	ý
13	CR	(Carriage return)	45	-	77	M	109	m	141	ì	173	¡	205	═	237	Ý
14	SO	(Shift Out)	46	.	78	N	110	n	142	Ä	174	«	206	╬	238	¯
15	SI	(Shift In)	47	/	79	O	111	o	143	Å	175	»	207	¤	239	´
16	DLE	(Data link escape)	48	0	80	P	112	p	144	É	176	░	208	ð	240	≡
17	DC1	(Device control 1)	49	1	81	Q	113	q	145	æ	177	▒	209	Ð	241	±
18	DC2	(Device control 2)	50	2	82	R	114	r	146	Æ	178	▓	210	Ê	242	‗
19	DC3	(Device control 3)	51	3	83	S	115	s	147	ô	179	│	211	Ë	243	¾
20	DC4	(Device control 4)	52	4	84	T	116	t	148	ö	180	┤	212	È	244	¶
21	NAK	(Negative acknowl.)	53	5	85	U	117	u	149	ò	181	Á	213	ı	245	§
22	SYN	(Synchronous idle)	54	6	86	V	118	v	150	û	182	Â	214	Í	246	÷
23	ETB	(End of trans. block)	55	7	87	W	119	w	151	ù	183	À	215	Î	247	¸
24	CAN	(Cancel)	56	8	88	X	120	x	152	ÿ	184	©	216	Ï	248	°
25	EM	(End of medium)	57	9	89	Y	121	y	153	Ö	185	╣	217	┘	249	¨
26	SUB	(Substitute)	58	:	90	Z	122	z	154	Ü	186	║	218	┌	250	·
27	ESC	(Escape)	59	;	91	[	123	{	155	ø	187	╗	219	▮	251	¹
28	FS	(File separator)	60	<	92	\	124	|	156	£	188	╝	220	▄	252	³

Bits

- To create a bit:

  bits = Bits(int=123,length=8)

  prints 0b01111011

- To create a bit array: 123

  barr = BitArray(num,8)

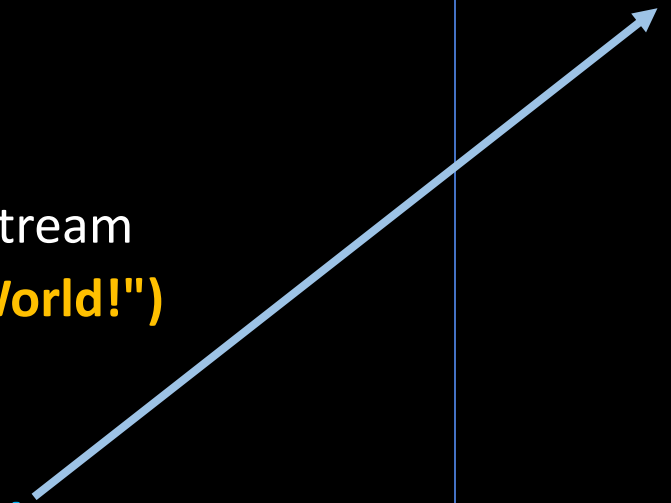
  prints 0b

0	1	1	1	1	0	1	1

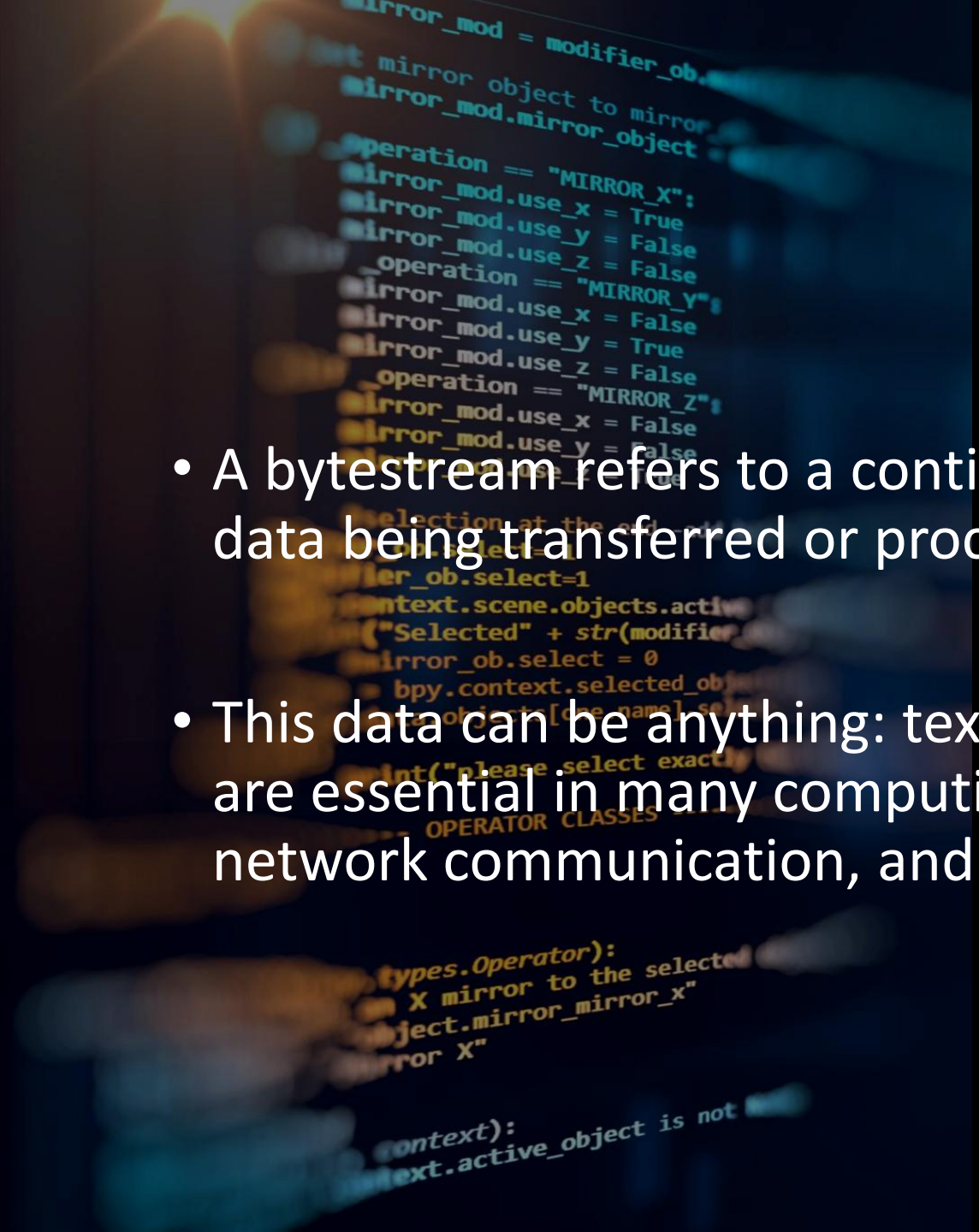
Bit Stream

```
from bitstream import BitStream
bits = BitStream(b"Hello World!")
print(bits)

0x486c656c6f20576f726c64
```



Binary	Hex	Ascii
01001000	0x48	H
01100101	0x65	e
01101100	0x6C	l
01101100	0x6C	l
01101111	0x6F	o
00100000	0x20	
01010111	0x57	W
01101111	0x6F	o
01110010	0x72	r
01101100	0x6C	l
01100100	0x64	d
00100001	0x21	!



Byte Stream

- A bytestream refers to a continuous sequence of bytes that represent data being transferred or processed.
- This data can be anything: text, images, audio, video, etc. Bytestreams are essential in many computing tasks, such as file I/O (input/output), network communication, and data serialization.

Byte Stream

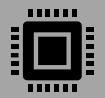
```
import requests
import os
req = requests.get("https://images.splash.com/") # example website
jpg = open("photo.jpg", "wb")
jpg.write(req.content)
```

d7 af 1f cf 3c f3 0c 6d da b4 61 ca 94 29 fc f6 db 6f 7c fc f1 c7 97 7d ce e9 d3 a7 8b 2e 16 49 4c cue 2c ba 6d d9 b2 85 b3 67 cf f2 c3 0f 3f 14 ed fb c1 07 1f b0 7b f7 6e d6 ac 59 c3 92 25 4b d7 af 1f cf 3c f3 0c 6d da b4 61 ca 94 29 fc f6 db 6f 7c fc f1 c7 97 7d ce e9 d3 a7 8b 2e 16 49 4c be 2c ba 6d d9 b2 85 b3 67 cf f2 c3 0f 3f 14 ed fb c1 07 1f b0 7b f7 6e d6 ac 59 c3 92 25 4b …

Serialization



Pickle is used for serializing and de-serializing Python object structures.



Serialization refers to the process of converting an object in memory to a byte stream.



The byte stream can be written to disk or broadcast to a network.



The byte stream can de-serialized back to a Python object. this character stream can then be retrieved and de-serialized back to a Python object.

Serialize - Dump

```
import pickle

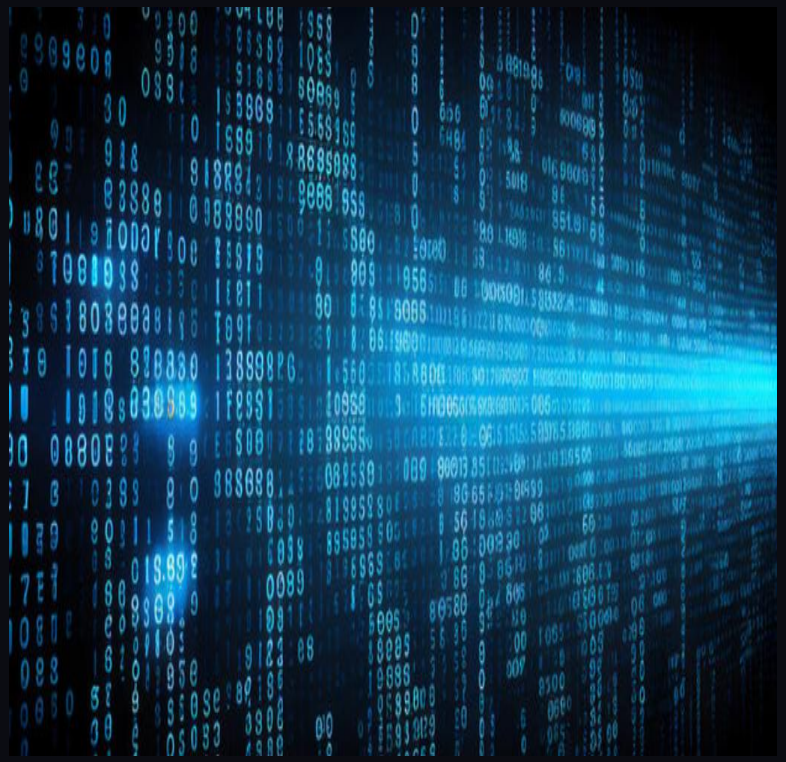
exampleDict = {'Python':3,’Java’:8,'Windows':10}
fileObj = open('data.obj', 'wb')
pickle.dump(exampleDict,fileObj)
fileObj.close()
```

De-Serialize Load

```
import pickle

fileObj = open('data.obj', 'rb')
exampleObj = pickle.load(fileObj)
fileObj.close()
```

Python Streams: the io Module



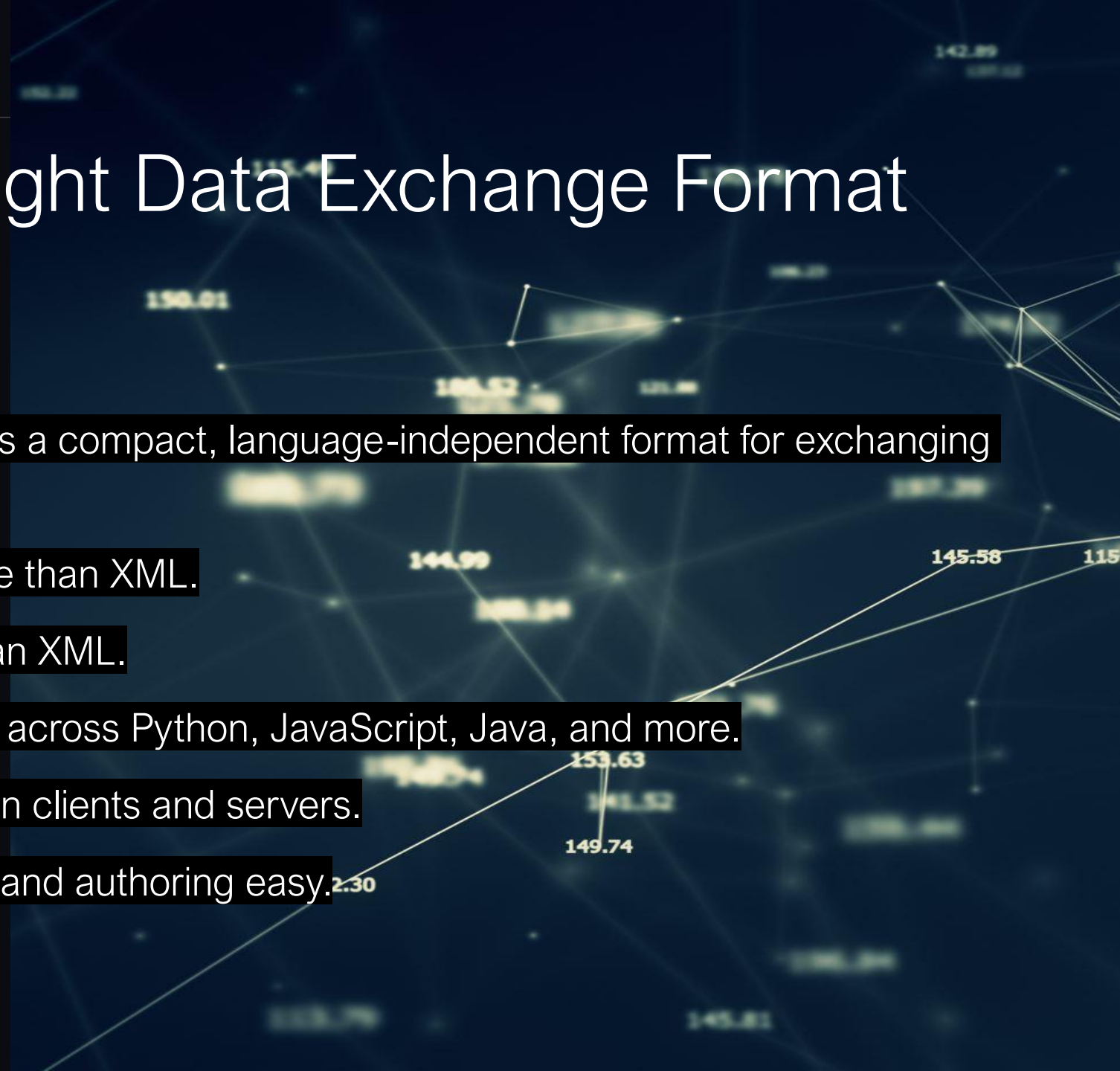
Python layers every stream in three tiers, all exposed by open().

Raw: io.FileIO unbuffered bytes straight from the OS file descriptor.

Buffered: BufferedReader, BufferedWriter, BufferedRandom batch syscalls through an in-memory buffer.

Text: io.TextIOWrapper wraps a buffered byte stream and handles encoding and newline translation.

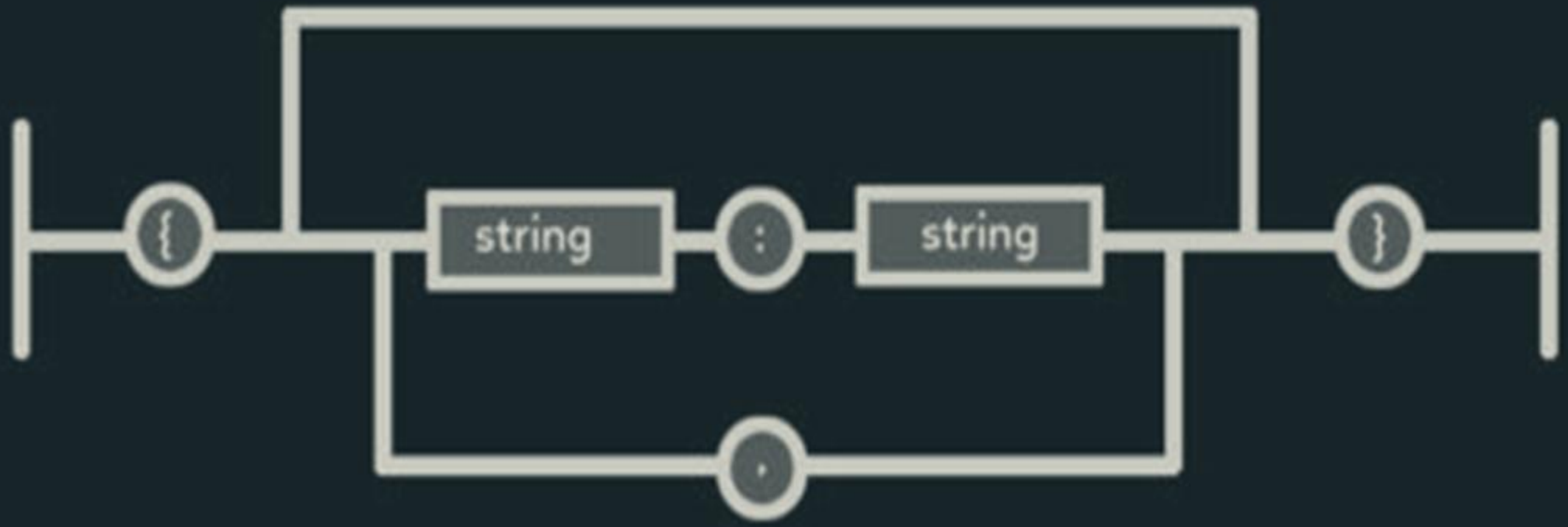
JSON: A Lightweight Data Exchange Format



JSON (JavaScript Object Notation) is a compact, language-independent format for exchanging data. Its strengths:

- Simple — easier to read and write than XML.
- Lightweight — more compact than XML.
- Language-independent — works across Python, JavaScript, Java, and more.
- Built for data interchange between clients and servers.
- Intuitive structure makes parsing and authoring easy.

JSON Object Data Format



Syntax

Objects are key-value pairs wrapped in curly braces { }.

Keys are strings; values can be string, number, object, array, true, false, or null.

Separate pairs with commas.

```
{
    "Planet": "Neptune",
    "Distance (AU)": "30.05",
    "Orbital Period (Years)": "164.8"
}
```

JSON

```
{
    "planet" :"Saturn",
    "distance_au": 9.58,
    "orbital_period_years": 29.46
}
```

HTML

```
<tr>
    <td>Saturn</td>
    <td>9.58</td>
    <td>29.46</td>
</tr>
```

XML

```
<planet_info>
    <planet>Saturn</planet>
    <distance_au>9.58</distance_au>
    <orbital_period_years>29.46</orbital_period_years>
</planet_info>
```